

Diffusion and Flow Matching: From a perspective of differential equations

Li Ju

li.ju@it.uu.se

1. FLOW MATCHING

Imagine a group of particles x , at time $t = 0$ they are denoted as x_0 and at time $t = 1$ as x_1 . Their movement along the time t is governed by an ordinary differential equation

$$\frac{dx_t}{dt} = v_t(x_t),$$

where v_t is a time-dependent vector field, which describes the velocity of the particle x at time t . Following the velocity field v_t , at anytime, x_t is also a random variable following some kind of distribution.

x_0 could be a simple and trivial distribution. What we are interested in is x_1 , which can be arbitrary complex. We want to design a probability path p_t that makes x_0 evolve to x_1 . But how do we find the velocity field v_t given p_t ? Continuity equation is the bridge.

1.1 Continuity Equation

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot J(x, t) \quad \text{where} \quad J(x, t) = p_t(x)v_t(x)$$

- $\frac{\partial p_t(x)}{\partial t}$: This is how probability density changes at location x at time goes forward.
- $J(x, t) = p_t(x)v_t(x)$: This is the flux of probability at location x , denoting how probability mass is passing through at x and at what direction.
- $\nabla_x \cdot J(x, t)$: The divergence of the flux, measuring the net outflow of probability mass at location x at time t .

Now we have the core tool to help us derive the velocity.

1.2 The conditionalization trick

The continuity equation looks like it finishes the job: choose a path p_t , read off a v_t that generates it. But we cannot choose p_t , because we do not know where it has to end. The target $p_1 = p_{\text{data}}$ is available to us only through samples, i.e., we have no formula for its density. So we have no formula for any path interpolating to it, and therefore nothing to substitute into the continuity equation.

What we *can* write down is a path to a **single data point**. Fix x_1 . The conditional path $p_t(\cdot \mid x_1)$ is entirely ours to design, subject only to the boundary conditions

$$p_0(x \mid x_1) = p_0(x) \quad (\text{independent of } x_1), \quad p_1(x \mid x_1) = \delta(x - x_1),$$

and for a sensible choice (say a Gaussian whose mean slides from 0 to x_1) the conditional velocity $v_t(x \mid x_1)$ comes out in closed form.

The idea of the conditionalization trick is to never construct the marginal path at all, but to *define* it as the mixture of these per-sample paths. The boundary conditions then hand us the endpoints for free:

$$p_0(x) = \int p_0(x)p_1(x_1)dx_1 = p_0(x), \quad p_1(x) = \int \delta(x - x_1)p_1(x_1)dx_1 = p_{\text{data}}(x),$$

and at no point did we need the density of p_{data} — only the ability to draw samples from it. That settles the *path*. It leaves the question of the *velocity*: the network has to be trained against the velocity field generating the marginal p_t , and so far we only control the conditional ones. Relating the two is what the next section does.

1.3 The Conditionalization trick on the velocity field

Given the joint distribution of x_t and x_1 , we have

$$p_t(x) = \int p_t(x, x_1) dx_1 = \int p_t(x | x_1) p_1(x_1) dx_1.$$

This shows that the marginal probability path $p_t(x)$ can be obtained from the marginalization of x_1 at any time x_t . What about the connection between the velocity field that transports p_0 to the marginal p_1 , and the one that transports p_0 to a specific x_1 ?

Starting from the continuity equation for the conditional path $p_t(x | x_1)$, we have

$$\begin{aligned} \frac{\partial p_t(x | x_1)}{\partial t} &= -\nabla_x \cdot (p_t(x | x_1) v_t(x | x_1)) \\ \int \frac{\partial p_t(x | x_1)}{\partial t} p_1(x_1) dx_1 &= -\int \nabla_x \cdot (p_t(x | x_1) v_t(x | x_1)) p_1(x_1) dx_1 \\ \frac{\partial p_t(x)}{\partial t} &= -\nabla_x \cdot \int p_t(x, x_1) v_t(x | x_1) dx_1 \\ \frac{\partial p_t(x)}{\partial t} &= -\nabla_x \cdot \left(p_t(x) \int p_t(x_1 | x) v_t(x | x_1) dx_1 \right) \end{aligned}$$

Comparing with the continuity equation of the marginal probability path, we see that

$$v_t(x) = \int p_t(x_1 | x) v_t(x | x_1) dx_1 = \mathbb{E}_{x_1 \sim p_t(x_1 | x)} [v_t(x | x_1)]$$

is a valid velocity field for p_t .

What this identity does and does not give us.

It is structural, not computational. It does not let us evaluate $v_t(x)$, because the posterior it averages against,

$$p_t(x_1 | x) = \frac{p_t(x | x_1) p_{\text{data}}(x_1)}{\int p_t(x | x'_1) p_{\text{data}}(x'_1) dx'_1},$$

is built from precisely the density we do not have, and would require an integral over the entire data distribution for every single x . What we have gained instead is a *characterisation* of the unknown target: the marginal velocity is the conditional expectation, given $x_t = x$, of a random quantity we can sample cheaply. This is the hinge of the next section.

1.4 Derivation of the loss function

If we are able to compute the true velocity field $v_t(x)$, we can use a neural network $v_\theta(x, t)$ to approximate it using the loss as follows

$$\ell_{\text{FM}}(\theta) = \mathbb{E}_{x \sim p_t(x), t \sim U(0,1)} [\|v_\theta(x, t) - v_t(x)\|^2]$$

But this loss cannot be evaluated: by the previous section $v_t(x)$ is a posterior expectation, and computing it at even one point x means integrating against $p_t(x_1 | x)$ over the whole data distribution. The way out is not to compute the target but to *replace it with a single noisy sample of itself*.

The conditional velocity $v_t(x | x_1)$ is available in closed form, and drawing $x_1 \sim p_1$ then $x \sim p_t(x | x_1)$ is exactly sampling from the pair (x, x_1) whose posterior appears above. This gives us the conditional flow matching loss

$$\ell_{\text{CFM}} = \mathbb{E}_{x_1 \sim p_1, x \sim p_t(x | x_1), t \sim U(0,1)} [\|v_\theta(x, t) - v_t(x | x_1)\|^2].$$

Now the next step is that we show that both losses are equivalent in the optimization.

$$\begin{aligned}
\ell_{\text{CFM}} &= \mathbb{E}_{x_1, x, t} [\|v_\theta(x, t)\|^2 - 2v_\theta(x, t)^\top v_t(x | x_1) + \|v_t(x | x_1)\|^2] \\
&= \mathbb{E}_{t, x \sim p_t(x)} \mathbb{E}_{x_1 \sim p_t(x_1|x)} [\|v_\theta(x, t)\|^2 - 2v_\theta(x, t)^\top v_t(x | x_1) + \|v_t(x | x_1)\|^2] \\
&= \mathbb{E}_{t, x \sim p_t(x)} [\|v_\theta(x, t)\|^2] - 2\mathbb{E}_{t, x \sim p_t(x)} [v_\theta(x, t)^\top \underbrace{\mathbb{E}_{x_1 \sim p_t(x_1|x)} v_t(x | x_1)}_{\text{marginal velocity field}}] + \mathbb{E}_{t, x \sim p_t(x)} \mathbb{E}_{x_1 \sim p_t(x_1|x)} [\|v_t(x | x_1)\|^2] \\
&= \mathbb{E}_{t, x \sim p_t(x)} [\|v_\theta(x, t)\|^2] - 2\mathbb{E}_{t, x \sim p_t(x)} [v_\theta(x, t)^\top v_t(x)] + \underbrace{\mathbb{E}_{t, x \sim p_t(x)} \mathbb{E}_{x_1 \sim p_t(x_1|x)} [\|v_t(x | x_1)\|^2]}_{\text{const. w.r.t. } \theta} \\
&= \ell_{\text{FM}} + \text{Const.}
\end{aligned}$$

Thus, we have shown that the CFM loss is only different to the FM loss up to a constant, which is irrelevant to the optimization process.

Where the trick was actually used.

The entire proof is one application of a standard fact about least squares:

$$\arg \min_f \mathbb{E} \|f(X) - Y\|^2 = \mathbb{E}[Y | X].$$

Regressing on a *random* target recovers its conditional mean, so training on $v_t(x | x_1)$ drives $v_\theta(x, t)$ toward $\mathbb{E}_{x_1 \sim p_t(x_1|x)} [v_t(x | x_1)]$ — the underbraced term above. This is the only place the conditionalization trick enters, and it is what makes the argument worth anything: without it we would know that CFM converges to some conditional average, but not that this average generates p_t and hence transports p_0 to p_{data} . The trick is not a recipe for computing the velocity field; it is the certificate that the quantity regression hands us is the right one. The noise in the individual targets does not vanish at the optimum: the constant we discarded is exactly its variance, but it is zero-mean, so it perturbs the gradient without moving the minimiser.

A concrete instance.

The simplest choice is the linear interpolant: draw $x_0 \sim p_0$, $x_1 \sim p_1$ and set

$$x_t = (1 - t)x_0 + tx_1 \quad \implies \quad v_t(x_t | x_0, x_1) = \dot{x}_t = x_1 - x_0.$$

The conditional target is just the straight-line displacement, and the training loop reduces to: sample a pair, sample t , regress $v_\theta((1 - t)x_0 + tx_1, t)$ onto $x_1 - x_0$. More generally, Gaussian conditional paths $p_t(x | x_1) = \mathcal{N}(\alpha_t x_1, \sigma_t^2 I)$ give $x_t = \alpha_t x_1 + \sigma_t \epsilon$ with

$$v_t(x_t | x_1) = \dot{\alpha}_t x_1 + \dot{\sigma}_t \epsilon,$$

of which the linear interpolant is the case $\alpha_t = t$, $\sigma_t = 1 - t$ with $x_0 = \epsilon$.

2. DIFFUSION MODELS

A note on indexing. In the flow matching section x_0 was noise and x_1 was data. From here on the convention flips to match the standard forward-noising direction: $x_0 \sim p_{\text{data}}$ and $p_1 = p_{\text{base}}$, so time runs data $\rightarrow$ noise and the generative direction is the reverse time $\tau = 1 - t$.

Now we extend the ODE to its stochastic variant, stochastic differential equations (SDEs) as follows: $dx_t = f_t(x_t)dt + g_t dW$, where

- $f_t(x_t)$: drift term, which is similar to the velocity field in ODE
- dW : Wiener term, which is the source of the randomness in the movement.
- g_t : Scalar controlling the amount of noise injected at time t .

Via this SDE, a source distribution p_0 can also be transported to a target distribution p_1 through a probability path p_t . This probability path p_t can be uniquely determined by the Fokker-Planck equation.

2.1 Fokker-Planck Equation

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot J(x, t) \quad \text{where} \quad J(x, t) = \underbrace{f_t(x)p_t(x)}_{J_{\text{drift}}} + \underbrace{\left(-\frac{g_t^2}{2}\nabla_x p_t(x)\right)}_{J_{\text{diffusion}}}$$

- $J(x, t)$: The total flux, denoting how much probability mass is passing through the location x at time t .
- J_{drift} : The flux caused by the drift term (the same as the continuity equation).
- $J_{\text{diffusion}}$: The gradient flux introduced by the diffusion term. Particles naturally flow from the region of the high density to the region of the low density, depending on the gradient of the density. Note that $\nabla_x \cdot \left(-\frac{g_t^2}{2}\nabla_x p_t(x)\right) = -\frac{g_t^2}{2}\Delta p_t(x)$, i.e. this term alone is the heat equation.

Probability flow ODE of a SDE

This looks similar to the continuity equation, with an additional diffusion flux. We can make it even more explicitly.

$$\begin{aligned} J(x, t) &= f_t(x)p_t(x) - \frac{g_t^2}{2}\nabla_x p_t(x) \\ &= p_t(x) \left[f_t(x) - \frac{g_t^2}{2} \cdot \frac{\nabla_x p_t(x)}{p_t(x)} \right] \\ &= p_t(x) \left[f_t(x) - \frac{g_t^2}{2} \cdot \nabla_x \ln p_t(x) \right] \end{aligned}$$

So that we convert the SDE to its ODE form $\frac{dx}{dt} = f_t(x) - \frac{g_t^2}{2}\nabla_x \ln p_t(x)$

This is in the exactly the same form as the continuity equation for an ODE, which is called probability flow ODE. This essentially tells us that adding a Wiener term to an ODE is equivalent to subtracting $\frac{g_t^2}{2}\nabla_x \ln p_t(x)$ from the velocity field of the ODE.

ODE and its associated SDE

Assuming we have a velocity field $v_t(x)$ that forms the distribution path p_t via the ODE

$$\frac{dx}{dt} = v_t(x)$$

If we add a Wiener term to the ODE, we obtain a SDE in the following form

$$dx = v_t(x)dt + g_t dW$$

Transforming the SDE into its own ODE, we have

$$\frac{dx}{dt} = v_t - \frac{g_t^2}{2}\nabla_x \ln p_t(x)$$

Obviously with the additional diffusion term, it does not recover the distribution path p_t any more, since naturally the diffusion process tend to spread the probability mass.

So what is the correct SDE that recovers the probability path? We need the drift term that cancels the impact of the diffusion term:

$$f_t(x) = v_t(x) + \frac{g_t^2}{2}\nabla_x \ln p_t(x)$$

With this drift term, the probability flow ODE is then

$$\frac{dx}{dt} = v_t(x) + \frac{g_t^2}{2}\nabla_x \ln p_t(x) - \frac{g_t^2}{2}\nabla_x \ln p_t(x) = v_t(x)$$

So, given the ODE, its associated SDE is actually

$$dx = (v_t(x) + \frac{g_t^2}{2}\nabla_x \ln p_t(x))dt + g_t dW$$

Time reverse SDE

From the probability flow ODE of a SDE, we can reverse a SDE easily. With a forward SDE which has a probability path p_t with p_0 at $t = 0$ and p_1 at time $t = 1$, how can we find another SDE that reverse this process with $p = 1$ at $\tau = 0$ and p_0 at $\tau = 1$? Assume that the SDE is given as follows

$$dx = f_t(x)dt + g_t dW$$

with its probability flow ODE

$$\frac{dx}{dt} = f_t(x) - \frac{g_t^2}{2} \nabla_x \ln p_t(x).$$

It is easy to reverse the ODE with $\tau = 1 - t$:

$$\frac{dx}{d\tau} = -f_\tau(x) + \frac{g_\tau^2}{2} \nabla_x \ln p_\tau(x) = (-f_\tau(x) + g_\tau^2 \nabla_x \ln p_\tau(x)) - \frac{g_\tau^2}{2} \nabla_x \ln p_\tau(x)$$

Thus, the corresponding reverse SDE is then

$$dx = (-f_\tau(x) + g_\tau^2 \nabla_x \ln p_\tau(x))d\tau + g_\tau dW$$

2.2 Denoising Diffusion Models

Now we have a SDE to transport one distribution to another via a probability path p_t , can we design p_t and find the drift term $f_t(x)$ to do the transport, like we did for flow matching? Not with a well-behaved drift.

Why?

For flow matching, we used the conditionalization trick: instead of designing the marginal probability path $p_t(x)$ directly, we design the conditional probability paths $p_t(x | x_1)$ and move p_0 to the dirac distribution at x_1 . However for diffusion models, this is infeasible: To force the conditional distribution $p_1(x | x_1)$ to be a dirac distribution (with an infinitely large score function at $t = 1$), the drift term $f_t(x | x_1)$ has to be infinitely large to suppress the randomness from the inherent diffusion term $g_t dW$. So the noise-to-data conditionalization trick does not work here with a finite, tractable drift.

For diffusion model, for the *conditional* probability path, we can instead, design the destructive diffusion process from a dirac distribution at a data point from distribution p_{data} to a base distribution p_{base} . This process will form a marginal probability path p_t from p_{data} to p_{base} . Reversing this marginal probability path, we can obtain the probability path from p_{base} to p_{data} .

Forward: Conditional Probability Path

With $x_0 \sim p_{\text{data}}$, we can design corresponding $f_t(x | x_0)$ and g_t , such that the conditional probability path $p_t(x | x_0)$ has $p_0(\cdot | x_0) = \delta(x - x_0)$ and $p_1(\cdot | x_0) = p_{\text{base}}$. We assume g_t is a constant w.r.t. x_0 .

Forward: Marginal Probability Path

The marginal probability path p_t can be obtained by marginalization of x_0 over p_{data} at any time

$$p_t(x) = \int p_t(x | x_0) p_{\text{data}}(x_0) dx_0$$

We can verify that $p_1(x) = \int p_{\text{base}}(x) p_{\text{data}}(x_0) dx_0 = p_{\text{base}}(x)$ and $p_0(x) = \int \delta(x - x_0) p_{\text{data}}(x_0) dx_0 = p_{\text{data}}(x)$.

Since we defined the drift term $f_t(x | x_0)$ on the conditional probability path, they are analytically accessible for each x_0 . But the drift term $f_t(x)$ for the marginal probability path is not obvious.

$$\begin{aligned}
\frac{\partial p_t(x | x_0)}{\partial t} &= -\nabla_x \cdot \left(p_t(x | x_0) f_t(x | x_0) - \frac{g_t^2}{2} \nabla_x p_t(x | x_0) \right) \\
\int \frac{\partial p_t(x | x_0)}{\partial t} p_{\text{data}}(x_0) dx_0 &= -\int \nabla_x \cdot \left(p_t(x | x_0) f_t(x | x_0) - \frac{g_t^2}{2} \nabla_x p_t(x | x_0) \right) p_{\text{data}}(x_0) dx_0 \\
\frac{\partial p_t(x)}{\partial t} &= -\nabla_x \cdot \left[\int p_t(x, x_0) f_t(x | x_0) dx_0 - \frac{g_t^2}{2} \nabla_x \int p_t(x, x_0) dx_0 \right] \\
\frac{\partial p_t(x)}{\partial t} &= -\nabla_x \cdot \left[p_t(x) \int p_t(x_0 | x) f_t(x | x_0) dx_0 - \frac{g_t^2}{2} \nabla_x p_t(x) \right]
\end{aligned}$$

Compared to the Fokker-Planck equation, we see that

$$f_t(x) = \int p_t(x_0 | x) f_t(x | x_0) dx_0 = \mathbb{E}_{x_0 \sim p(x_0|x)}[f_t(x | x_0)]$$

So in principle the drift term for the marginal probability path $f_t(x)$ is intractable directly. But we have freedom in choosing the drift term, and if we design it to be x_0 -independent then $f_t(x) = f_t(x | x_0)$ and the expectation collapses.

This is why the standard choice is an **affine** drift, $f_t(x) = a_t x$ (e.g. the VP-SDE $f_t(x) = -\frac{1}{2}\beta_t x$ with $g_t = \sqrt{\beta_t}$). Affinity buys two things at once: it is manifestly independent of x_0 , so the identity above is satisfied trivially; and it keeps $p_t(x | x_0)$ Gaussian, $\mathcal{N}(\alpha_t x_0, \sigma_t^2 I)$, so the conditional score needed below is available in closed form,

$$\nabla_x \ln p_t(x | x_0) = -\frac{x - \alpha_t x_0}{\sigma_t^2} = -\frac{\epsilon}{\sigma_t}.$$

Reverse

Reversing the SDE for the marginal probability path $p_t(x)$, we have the following reverse SDE

$$dx = (-f_\tau(x) + g_\tau^2 \nabla_x \ln p_\tau(x)) d\tau + g_\tau dW$$

The drift term $-f_\tau$ and the scalar g_τ are known by design, the only unknown term here is the score function $\nabla_x \ln p_\tau(x)$. We can use a neural network $s_\theta(x, \tau)$ to approximate it

$$\ell_{\text{SM}} = \mathbb{E}_{\tau \sim U(0,1), x \sim p_\tau(x)} [\|s_\theta(x, \tau) - \nabla_x \ln p_\tau(x)\|^2]$$

But still the score term to be approximated is unknown.

Like we did in Flow Matching, is it possible to use the conditionalization trick on the score function

$$\ell_{\text{CSM}} = \mathbb{E}_{\tau \sim U(0,1), x_0 \sim p_{\text{data}}, x \sim p_\tau(x|x_0)} [\|s_\theta(x, \tau) - \nabla_x \ln p_\tau(x | x_0)\|^2]$$

But is ℓ_{CSM} equivalent to ℓ_{SM} ? Let's find out.

$$\begin{aligned}
\ell_{\text{CSM}} &= \mathbb{E}_{\tau \sim U(0,1), x_0 \sim p_{\text{data}}, x \sim p_\tau(x|x_0)} [\|s_\theta(x, \tau) - \nabla_x \ln p_\tau(x | x_0)\|^2] \\
&= \mathbb{E}_{\tau \sim U(0,1), x_0 \sim p_{\text{data}}, x \sim p_\tau(x|x_0)} [\|s_\theta(x, \tau)\|^2 - 2s_\theta(x, \tau)^\top \nabla_x \ln p_\tau(x | x_0)] + \text{const.} \\
&= \mathbb{E}_{\tau \sim U(0,1), x \sim p_\tau(x)} [\|s_\theta(x, \tau)\|^2] - 2\mathbb{E}_{\tau \sim U(0,1), x_0 \sim p_{\text{data}}, x \sim p_\tau(x|x_0)} [s_\theta(x, \tau)^\top \nabla_x \ln p_\tau(x | x_0)] + \text{const.} \\
&= \mathbb{E}_{\tau \sim U(0,1), x \sim p_\tau(x)} [\|s_\theta(x, \tau)\|^2] - 2\mathbb{E}_{\tau \sim U(0,1), x_0 \sim p_{\text{data}}} \mathbb{E}_{x \sim p_\tau(x|x_0)} \left[s_\theta(x, \tau)^\top \frac{\nabla_x p_\tau(x | x_0)}{p_\tau(x | x_0)} \right] + \text{const.} \\
&= \mathbb{E}_{\tau \sim U(0,1), x \sim p_\tau(x)} [\|s_\theta(x, \tau)\|^2] - 2\mathbb{E}_{\tau \sim U(0,1)} \iint s_\theta(x, \tau)^\top \nabla_x p_\tau(x | x_0) p_{\text{data}}(x_0) dx dx_0 + \text{const.} \\
&= \mathbb{E}_{\tau \sim U(0,1), x \sim p_\tau(x)} [\|s_\theta(x, \tau)\|^2] - 2\mathbb{E}_{\tau \sim U(0,1)} \int s_\theta(x, \tau)^\top \nabla_x \int p_\tau(x, x_0) dx_0 dx + \text{const.} \\
&= \mathbb{E}_{\tau \sim U(0,1), x \sim p_\tau(x)} [\|s_\theta(x, \tau)\|^2] - 2\mathbb{E}_{\tau \sim U(0,1)} \int s_\theta(x, \tau)^\top \frac{\nabla_x p_\tau(x)}{p_\tau(x)} p_\tau(x) dx + \text{const.} \\
&= \mathbb{E}_{\tau \sim U(0,1), x \sim p_\tau(x)} [\|s_\theta(x, \tau)\|^2] - 2\mathbb{E}_{\tau \sim U(0,1), x \sim p_\tau(x)} s_\theta(x, \tau)^\top \nabla_x \ln p_\tau(x) + \text{const.} \\
&= \ell_{\text{SM}} + \text{const.}
\end{aligned}$$

3. BRIDGING THE TWO: SCORE AND VELOCITY ARE THE SAME OBJECT

The two halves above were built around different learned quantities — a velocity field v_θ for flow matching, a score s_θ for diffusion. For Gaussian conditional paths they are related by an affine change of variables, so the two frameworks are reparametrizations of each other rather than genuinely different models.

Take $p_t(x \mid x_1) = \mathcal{N}(\alpha_t x_1, \sigma_t^2 I)$, i.e. $x_t = \alpha_t x_1 + \sigma_t \epsilon$. Both the conditional velocity and the conditional score are linear in (x_1, ϵ) :

$$v_t(x \mid x_1) = \dot{\alpha}_t x_1 + \dot{\sigma}_t \epsilon, \quad \nabla_x \ln p_t(x \mid x_1) = -\frac{\epsilon}{\sigma_t}.$$

Eliminating x_1 and ϵ between these and $x = \alpha_t x_1 + \sigma_t \epsilon$, and then taking $\mathbb{E}[\cdot \mid x_t = x]$ on both sides (both marginalizations are the same conditional expectation, as shown in the two derivations above), gives

$$v_t(x) = \frac{\dot{\alpha}_t}{\alpha_t} x + \left(\frac{\dot{\alpha}_t}{\alpha_t} \sigma_t^2 - \dot{\sigma}_t \sigma_t \right) \nabla_x \ln p_t(x),$$

which inverts to

$$\nabla_x \ln p_t(x) = \frac{v_t(x) - \frac{\dot{\alpha}_t}{\alpha_t} x}{\frac{\dot{\alpha}_t}{\alpha_t} \sigma_t^2 - \dot{\sigma}_t \sigma_t}.$$

As a check, the linear interpolant $\alpha_t = t$, $\sigma_t = 1 - t$ gives $v_t(x) = \frac{x}{t} + \frac{1-t}{t} \nabla_x \ln p_t(x)$. So a trained score network gives a velocity field for free and vice versa. This also closes the loop with the "ODE and its associated SDE" section: once either quantity is known, one can sample either deterministically via the probability flow ODE $\dot{x} = v_t(x)$, or stochastically via

$$dx = \left(v_t(x) + \frac{g_t^2}{2} \nabla_x \ln p_t(x) \right) dt + g_t dW$$

for *any* choice of noise schedule g_t , all sharing the same marginals p_t . The choice of g_t at sampling time is therefore a free knob trading off discretization error against error correction, not a property of the trained model.